Our HW1 Machine

Let's imagine we are going to work with an *invented* machine that follows the Von Neumann philosophy and will help us understand how a basic computer works and how it executes instructions. Keep in mind that our microprocessor, as we said, is fictional. This microprocessor does not exist; obviously, real processors are far more complex than what we are about to study. However, by simplifying it, we can better understand the concept and work with it in a more direct way.

First, let's look at the components our basic computer would have. We can distinguish the following:

1. CU (Control Unit). The Control Unit allows programs to be executed in our example microprocessor. It is responsible for activating the circuitry needed to perform the required operations.

2. Memory. It stores the program code we want to execute and its data.

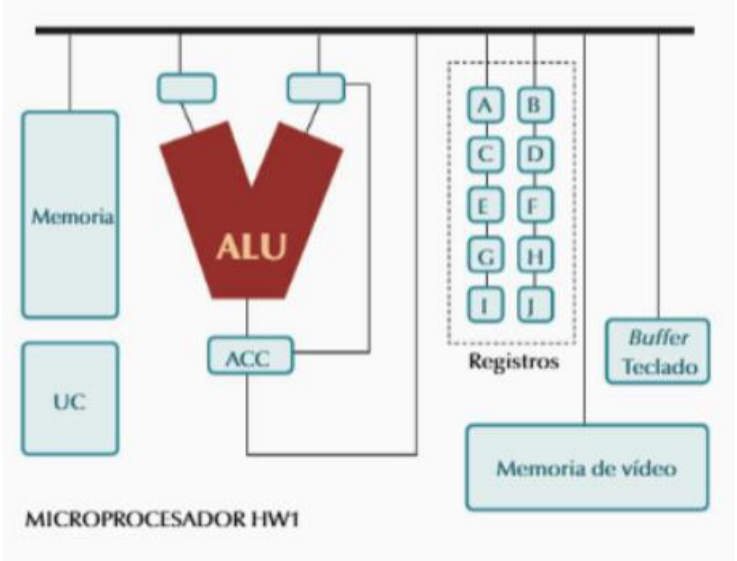
3. ALU (Arithmetic–Logic Unit). It is responsible for executing operations. It can take up to two operands, and the result of the operation is stored in a buffer called the accumulator or ACC. The content of the accumulator can be transferred to an operand.

4. ACC (Accumulator). A buffer where the results of executed operations are stored.

5. Registers (A–J). They allow us to store intermediate results of the operations carried out by our microprocessor.

6. Keyboard Buffer. It stores the keystrokes from a hypothetical keyboard. When a key is pressed, it is stored in the keyboard buffer, and this allows us to read and process those data later.

7. Video Memory. It allows us to communicate with the outside world. Any data written to video memory will appear on the screen.



Instructions of Our HW1 Machine

Arithmetic Operations

- **SUM ACC RX** This instruction adds the content of the accumulator and register **RX** (which may be **RA**, **RB**, etc.), and places the result back into the accumulator. Example: **SUM ACC RC**
- **SUB RX ACC** This instruction subtracts register **RX** from the content of the accumulator, and places the result back into the accumulator. Example: **SUB RC ACC**

Keyboard Input

- **READ RX** This instruction reads a value from the keyboard and stores it in the specified register.

Screen Output Instructions

- **PRINT RX** This instruction displays on screen the content of the register we specify (register **RX** — where X may be A, B… J).

- **PRINT "TEXT"** This instruction displays on screen the text we choose. For example: **PRINT "End of execution"** will show on screen: **End of execution**

Assignment Instructions

- **MOVE RX ACC** This instruction copies the content of the accumulator into the specified register (register RX, where X may be A, B… J).
- **MOVE ACC RX** This instruction copies the content of a register into the accumulator.
- **MOVE ACC DATA** This instruction places a specific value into the accumulator. For example: MOVE ACC 5 will place the number 5 in the accumulator.

Jump Instructions

- **JMP IF ZERO INSTX** These jump instructions allow breaking the sequential flow of execution. This instruction jumps to instruction X if the content of the accumulator is 0. Example: JMP IF ZERO 15 jumps to instruction 15 if the accumulator contains 0.
- **JMP IF NOT ZERO INSTX** This instruction jumps to instruction X if the content of the accumulator is not 0. Example: JMP IF NOT ZERO 15 jumps to instruction 15 if the accumulator is not 0.
- **JMP INSTX** This instruction jumps to instruction X, regardless of the accumulator's content.
- **INSTX**

Tasks

1. Create a program using HW1 machine instructions that reads two values from the keyboard and displays their sum on the screen.

2. This time, create a program using HW1 machine instructions that reads three values from the keyboard and displays their sum on the screen.

3. Create a program using HW1 machine instructions that displays the numbers from 1 to 10 on the screen.